

Project 3

Aarav Chauhan – 2501420027

1) Bit

```
2) CHIP Bit {
3)     IN in, load;
4)     OUT out;
5)
6)     PARTS:
7)     Mux(a=outLoop , b=in , sel=load , out=muxOut );
8)     DFF(in=muxOut , out=outLoop, out=out );
9) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The main window shows the HDL code for the Bit chip, which implements a 1-bit register using a multiplexer and a D flip-flop. The code is as follows:

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/3/a/Bit.hdl
5 /**
6  * 1-bit register:
7  * If load is asserted, the register's value is set to in;
8  * Otherwise, the register maintains its current value:
9  * if (load(t)) out(t+1) = in(t), else out(t+1) = out(t)
10 */
11 CHIP Bit {
12     IN in, load;
13     OUT out;
14
15     PARTS:
16     Mux(a=outLoop , b=in , sel=load , out=muxOut );
17     DFF(in=muxOut , out=outLoop, out=out );
18 }
```

The right-hand panel shows the chip's configuration:

- Input pins:** in (1), load (0)
- Output pins:** out (0)
- Internal pins:** outLoop (0), muxOut (0)

The bottom panel shows the test script and simulation results:

```
Test Script
855 tock,
856 output;
857
858 set in 1,
859 set load 0,
860 tick,
861 output;
862
863 tock,
864 output;
```

Simulation successful: The output file is identical to the compare file

2) Register

```
3) CHIP Register {
4)     IN in[16], load;
5)     OUT out[16];
6)
7)     PARTS:
8)     Bit(in=in[0] , load=load , out=out[0] );
9)     Bit(in=in[1] , load=load , out=out[1] );
10)         Bit(in=in[2] , load=load , out=out[2] );
11)         Bit(in=in[3] , load=load , out=out[3] );
12)         Bit(in=in[4] , load=load , out=out[4] );
13)         Bit(in=in[5] , load=load , out=out[5] );
14)         Bit(in=in[6] , load=load , out=out[6] );
15)         Bit(in=in[7] , load=load , out=out[7] );
16)         Bit(in=in[8] , load=load , out=out[8] );
17)         Bit(in=in[9] , load=load , out=out[9] );
18)         Bit(in=in[10] , load=load , out=out[10] );
19)         Bit(in=in[11] , load=load , out=out[11] );
20)         Bit(in=in[12] , load=load , out=out[12] );
21)         Bit(in=in[13] , load=load , out=out[13] );
22)         Bit(in=in[14] , load=load , out=out[14] );
23)         Bit(in=in[15] , load=load , out=out[15] );
24) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the project name "NAND2Tetris / Hardware Simulator / Register.hdl". The main editor area shows the HDL code for the Register chip, which is a 16-bit register. The code defines an input array 'in' of size 16, an output array 'out' of size 16, and a 'load' input. It uses 16 instances of the 'Bit' component to connect each input bit to its corresponding output bit. The right-hand side of the simulator shows the 'Chip Register' configuration. The 'Input pins' section shows 'in' as a 16-bit bus with the value '0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1' and 'load' as a single-bit input with the value '1'. The 'Output pins' section shows 'out' as a 16-bit bus with the same value '0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1'. The bottom status bar indicates 'Test Script' is active, showing a sequence of commands: '559', '560 tock,', '561 output;', '562', '563 set load 1,', '564 tick,', '565 output;', '566', '567 tock,', '568 output;'. A green message at the bottom states 'Simulation successful: The output file is identical to the compare file'.

3) Ram8

```
4) CHIP RAM8 {
5)     IN in[16], load, address[3];
6)     OUT out[16];
7)
8)     PARTS:
9)     DMux8Way(in=load , sel=address , a=x0 , b=x1 , c=x2 , d=x3
    , e=x4 , f=x5 , g=x6 , h=x7 );
10)     Register(in=in , load=x0 , out=y0 );
11)     Register(in=in , load=x1 , out=y1 );
12)     Register(in=in , load=x2 , out=y2 );
13)     Register(in=in , load=x3 , out=y3 );
14)     Register(in=in , load=x4 , out=y4 );
15)     Register(in=in , load=x5 , out=y5 );
16)     Register(in=in , load=x6 , out=y6 );
17)     Register(in=in , load=x7 , out=y7 );
18)     Mux8Way16(a=y0 , b=y1 , c=y2 , d=y3 , e=y4 , f=y5 ,
    g=y6 , h=y7 , sel=address , out=out );
19) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The main window shows the HDL code for the RAM8 chip, which is a Verilog implementation of an 8-word RAM. The code defines an 8-to-1 multiplexer (DMux8Way) and eight registers, each loaded with a bit of the input data and selected by a bit of the address. The outputs of these registers are then multiplexed back together to form the 16-bit output data.

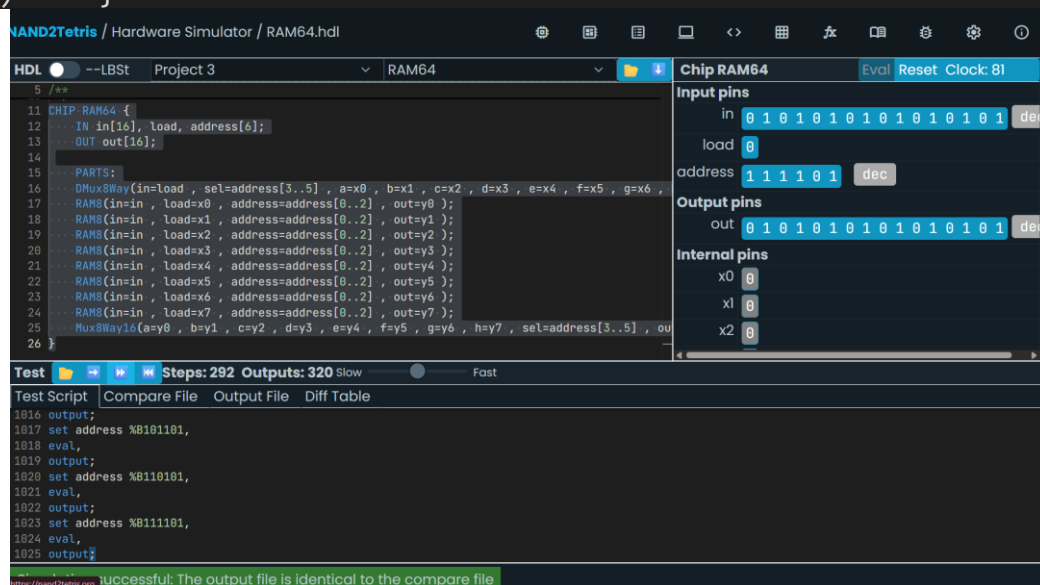
On the right side, the 'Chip RAM8' configuration panel is visible, showing the input pins (in, load, address) and output pins (out). The internal pins (x0, x1, x2) are also shown. The 'Eval' button is highlighted, indicating that the simulation has been executed.

At the bottom, the 'Test' panel shows the test script, which sets the address to 5, evaluates the chip, and outputs the result. The test results show that the output is identical to the compare file, confirming the correctness of the implementation.

Simulation successful: The output file is identical to the compare file

4) Ram64

```
5) CHIP RAM64 {
6)     IN in[16], load, address[6];
7)     OUT out[16];
8)
9)     PARTS:
10)         DMux8Way(in=load , sel=address[3..5] , a=x0 , b=x1 ,
            c=x2 , d=x3 , e=x4 , f=x5 , g=x6 , h=x7 );
11)         RAM8(in=in , load=x0 , address=address[0..2] , out=y0
            );
12)         RAM8(in=in , load=x1 , address=address[0..2] , out=y1
            );
13)         RAM8(in=in , load=x2 , address=address[0..2] , out=y2
            );
14)         RAM8(in=in , load=x3 , address=address[0..2] , out=y3
            );
15)         RAM8(in=in , load=x4 , address=address[0..2] , out=y4
            );
16)         RAM8(in=in , load=x5 , address=address[0..2] , out=y5
            );
17)         RAM8(in=in , load=x6 , address=address[0..2] , out=y6
            );
18)         RAM8(in=in , load=x7 , address=address[0..2] , out=y7
            );
19)         Mux8Way16(a=y0 , b=y1 , c=y2 , d=y3 , e=y4 , f=y5 ,
            g=y6 , h=y7 , sel=address[3..5] , out=out );
20) }
```



5) Ram512

```
6) CHIP RAM512 {
7)     IN in[16], load, address[9];
8)     OUT out[16];
9)
10)     PARTS:
11)         DMux8Way(in=load , sel=address[6..8] , a=x0 , b=x1 ,
            c=x2 , d=x3 , e=x4 , f=x5 , g=x6 , h=x7 );
12)         RAM64(in=in , load=x0 , address=address[0..5] ,
            out=y0 );
13)         RAM64(in=in , load=x1 , address=address[0..5] ,
            out=y1 );
14)         RAM64(in=in , load=x2 , address=address[0..5] ,
            out=y2 );
15)         RAM64(in=in , load=x3 , address=address[0..5] ,
            out=y3 );
16)         RAM64(in=in , load=x4 , address=address[0..5] ,
            out=y4 );
17)         RAM64(in=in , load=x5 , address=address[0..5] ,
            out=y5 );
18)         RAM64(in=in , load=x6 , address=address[0..5] ,
            out=y6 );
19)         RAM64(in=in , load=x7 , address=address[0..5] ,
            out=y7 );
20)         Mux8Way16(a=y0 , b=y1 , c=y2 , d=y3 , e=y4 , f=y5 ,
            g=y6 , h=y7 , sel=address[6..8] , out=out );
21) }
```

The screenshot displays the Nand2Tetris Hardware Simulator interface. The main window shows the HDL code for the RAM512 chip, which is a Verilog-like description of a 512-bit RAM. The code is organized into sections: a chip definition, input/output pins, a parts section for internal components, and a final closing brace. The internal components include a DMux8Way and eight RAM64 chips, each loaded with a different bit of the 8-bit load signal. The output is a 16-bit Mux8Way16 that selects one of the eight 8-bit outputs based on the 3-bit select signal.

On the right side, the 'Chip RAM512' panel shows the current state of the chip. The input pins are 'in' (16 bits, 0101010101010101), 'load' (1 bit, 0), and 'address' (9 bits, 111101010). The output pin is 'out' (16 bits, 0101010101010101). The internal pins are 'x0' through 'x7' (all 0) and 'y0' through 'y7' (all 0).

At the bottom, the 'Test' panel shows the simulation progress: 'Steps: 292 Outputs: 320'. Below this, a 'Test Script' section shows a series of commands to set the address and output, and a final 'Simulation successful: The output file is identical to the compare file' message.

6) Ram4k

```
7) CHIP RAM4K {
8)     IN in[16], load, address[12];
9)     OUT out[16];
10)
11)     PARTS:
12)         DMux8Way(in=load , sel=address[9..11] , a=x0 , b=x1 ,
            c=x2 , d=x3 , e=x4 , f=x5 , g=x6 , h=x7 );
13)         RAM512(in=in , load=x0 , address=address[0..8] ,
            out=y0 );
14)         RAM512(in=in , load=x1 , address=address[0..8] ,
            out=y1 );
15)         RAM512(in=in , load=x2 , address=address[0..8] ,
            out=y2 );
16)         RAM512(in=in , load=x3 , address=address[0..8] ,
            out=y3 );
17)         RAM512(in=in , load=x4 , address=address[0..8] ,
            out=y4 );
18)         RAM512(in=in , load=x5 , address=address[0..8] ,
            out=y5 );
19)         RAM512(in=in , load=x6 , address=address[0..8] ,
            out=y6 );
20)         RAM512(in=in , load=x7 , address=address[0..8] ,
            out=y7 );
21)         Mux8Way16(a=y0 , b=y1 , c=y2 , d=y3 , e=y4 , f=y5 ,
            g=y6 , h=y7 , sel=address[9..11] , out=out );
22) }
```

The screenshot displays the Nand2Tetris Hardware Simulator interface. The top bar shows the project name "NAND2Tetris / Hardware Simulator / RAM4K.hdl". The main editor window shows the Verilog code for the RAM4K chip, which is a 4Kbit RAM implemented using 8 512-bit RAMs and an 8-to-16 multiplexer. The code includes a DMux8Way component to select one of the 8 RAMs based on the address bits [9..11], and a Mux8Way16 component to combine the outputs of the 8 RAMs into a single 16-bit output based on the address bits [9..11].

On the right side, the "Chip RAM4K" panel shows the input and output pins. The input pins are "in" (16 bits), "load" (1 bit), and "address" (12 bits). The output pin is "out" (16 bits). The internal pins are "x0" through "x7" (8 bits each) and "y0" through "y7" (8 bits each). The "load" pin is set to 0, and the "address" pin is set to 111101010101.

At the bottom, the "Test" panel shows the simulation results. The test script includes commands to set the address and output, and the simulation is successful, indicating that the output file is identical to the compare file.

7) Ram16k

```
8) CHIP RAM16K {
9)     IN in[16], load, address[14];
10)     OUT out[16];
11)
12)     PARTS:
13)     DMux4Way(in=load , sel=address[12..13] , a=x0 , b=x1
    , c=x2 , d=x3 );
14)     RAM4K(in=in , load=x0 , address=address[0..11] ,
    out=y0 );
15)     RAM4K(in=in , load=x1 , address=address[0..11] ,
    out=y1 );
16)     RAM4K(in=in , load=x2 , address=address[0..11] ,
    out=y2 );
17)     RAM4K(in=in , load=x3 , address=address[0..11] ,
    out=y3 );
18)     Mux4Way16(a=y0 , b=y1 , c=y2 , d=y3 ,
    sel=address[12..13] , out=out );
19) }
```

NAND2Tetris / Hardware Simulator / RAM16K.hdl

HDL ☒ --LBSt Project 3 RAM16K

Chip RAM16K Eval Reset Clock: 81

Input pins

in 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec

load 0

address 1 1 1 1 0 1 0 1 0 1 0 1 0 1 dec

Output pins

out 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec

Internal pins

x0 0

x1 0

x2 0

Test ☒ ☐ ☐ Steps: 292 Outputs: 320 Slow Fast

Test Script Compare File Output File Diff Table

```
1016 output;
1017 set address %B10110101010101,
1018 eval,
1019 output;
1020 set address %B11010101010101,
1021 eval,
1022 output;
1023 set address %B11110101010101,
1024 eval,
1025 output;
```

Simulation successful: The output file is identical to the compare file

8) PC

```
9) CHIP PC {
10)     IN in[16], reset, load, inc;
11)     OUT out[16];
12)
13)     PARTS:
14)     Inc16(in=regout, out=incout);
15)
16)     Mux16(a=regout, b=incout, sel=inc, out=mux1out);
17)
18)     Mux16(a=mux1out, b=in, sel=load, out=mux2out);
19)
20)     Mux16(a=mux2out, b=false, sel=reset, out=mux3out);
21)
22)     Register(in=mux3out, load=true, out=out, out=regout);
23) }
```

The screenshot displays the HDLBits Hardware Simulator interface for the 'PC.hdl' file. The top bar shows the project name 'PC' and the file 'PC.hdl'. The main editor shows the HDL code for the 'CHIP PC' component, which is a 16-bit processor. The code includes input pins for 'in[16]', 'reset', 'load', and 'inc', and an output pin for 'out[16]'. The logic consists of a 16-bit register, a 16-bit incrementer, and three 16-bit multiplexers. The 'regout' pin is connected to the output of the register, and the 'incout' pin is connected to the output of the incrementer. The 'mux1out' pin is connected to the output of the first multiplexer, and the 'mux2out' pin is connected to the output of the second multiplexer. The 'mux3out' pin is connected to the output of the third multiplexer. The 'out' pin is connected to the output of the register.

The right panel shows the 'Chip PC' configuration. It includes input pins: 'in' (16 bits, value 0101011011001110), 'reset' (1 bit, value 1), 'load' (1 bit, value 0), and 'inc' (1 bit, value 0). It also shows output pins: 'out' (16 bits, value 0000000000000000). Internal pins are also shown: 'regout' (16 bits, value 0000000000000000) and 'incout' (16 bits, value 0000000000000001).

The bottom panel shows the 'Test' script, which includes a 'Test Script' tab and a 'Test' button. The script contains the following code:

```
115 output;
116
117 set in 22222,
118 set reset 1,
119 set inc 0,
120 tick,
121 output;
122
123 tock,
124 output;
```

A green status bar at the bottom indicates: 'Simulation successful: The output file is identical to the compare file'.